Self-Training with Structured Data for Efficient Insect Pest Detection

Bachelor's Thesis

Christoph Wald
Matrikelnummer 17515905
christoph.wald@stud.uni-goettingen.de

Supervisors:
Prof. Alexander Ecker (Georg-August-Universität Göttingen)
Dr. Elias Böckmann (Julius-Kühn-Institut Braunschweig)

November 20, 2025

Contents

1 Introduction

Insect pests are responsible for significant production losses in agriculture [1]. However, pesticides threaten the environment by reducing biodiversity [2], particularly among insects, including pollinators that support agricultural production [3]. Overuse of pesticides causes resistance in pests. Additionally, global warming leads to increasing pest populations, which aggravates the problem further [1]. To address these challenges, the concept of integrated pest management (IPM) was developed. As defined by the Food and Agriculture Organization of the United Nations (FAO), it sets the goal of minimizing the use of pesticides by applying alternative management strategies wherever possible [4,5]. IPM is also the legal basis for the use of pesticides in the EU [6], which, among other things, prescribes the monitoring of insect pests when using pesticides. Precise monitoring facilitates the early and targeted use of pesticides for only small infested patches, the use of less harmful pesticides, or alternative ways of control like beneficial insects. Monitoring is usually based on setting up a grid of sticky traps, which are regularly checked for trapped insects. Because the manual counting requires expertise and takes a lot of time, it is not feasible for larger applications [7]. Hence, computer vision-based systems for automated monitoring of insect pests are considered to be part of the solution. Ongoing efforts to automatize insect pest monitoring have more recently adopted the methods of deep learning (see the surveys Teixeira et al. 2023 [8] and Mittal et al. 2024 [9]). My thesis tests a method to facilitate the implementation of deep learning-based, automated insect pest monitoring.

The dominant approach in pest monitoring in general involves the supervised training of YOLO models. YOLO ("You Only Look Once") is a deep learning architecture that unifies the tasks of object localization and classification into a single step. It was introduced by Redmon et al. in 2016 [10] and has been improved on since then in several iterations by different contributors. For greenhouses, specifically designed neural network models have been employed successfully in 2021 [11,12], but the recent studies from 2023 onward use YOLO models on individual datasets [13,14,15,16]. These six papers all report high performance, with mean average precision (mAP) or F1 scores ranging from 0.89 to 0.94.

Main challenges in the implementation are the small size of pests—which, even with high-resolution cameras, occupy only a very tiny fraction of an image—and the high degree of morphological variation among individuals. In less controlled environments additional complications arise from high morphological similarity between different species. These challenges create a demand for large annotated datasets; however, the collection and labeling of biological data require considerable effort and expertise. As a result, datasets are often small and exhibit other limitations, such as class imbalance. For an overview of

these specific challenges, see Teixeira et al. (2023) [8]. As a remedy, the authors recommend semi-supervised learning (SSL), a family of methods for training with only partially data. Despite this, no survey reports applications of SSL in the field of pest monitoring in general and only one study in the context of the greenhouse makes use of SSL methods: Rustia et al. [17] proposed a self-training pipeline as a supplement to a fully supervised model as early as 2021. While the results were promising, the implementation was highly intricate, involving multi-stage setups with hand-tailored solutions. This complexity may explain why the approach has not been further explored. Outside of the greenhouse, in open-field settings, some efforts were made from 2024 onward, either focusing on a single insect species with more diverse backgrounds or on a set of larger insects [18,19,20,21,22]. Another notable paper in this context is Dang et al. (2024) [23], who employ a field dataset of ten larger insect species and perform segmentation without manual annotation by leveraging a grounding language–image model. While extending this approach with classification and adapting it to the smaller insect pests typically found in greenhouses might be feasible, grounding models will not be considered in this study due to their limited speed and efficiency compared to YOLO-based methods.

Table 1 below shows an overview over the relevant literature. Green indicates a correspondence with the demands of my project, red the opposite, and orange overlappings. Because of the differing datasets and metrics, the results are not directly comparable. Also not seen in the table are the pest classes included in the datasets: Only Rustia et. al [11] covered the same pest classes featured in this project. As of now no YOLO model has been trained to identify common greenhouse pests without relying on supervised training. Although working solutions for automatic insect pest monitoring in the greenhouse have been successfully tested, practical deployment stagnates and is hindered by the need for large, manually annotated datasets for new target domains. To my knowledge, there is no adequate method to reduce the amount of labels needed. The goal of the thesis is therefore an improvement in implementation efficiency by adapting the labeling and training method.

In this thesis I test the proposition that given a specifically structured training dataset, a model can be trained with manual labels needed only for validation and testing. This should be achieved by a combination of image processing and a semi-supervised learning method (SSL) called self-training. The key requirement is the specific structure of the dataset, which should be partitioned into subsets. All images in one subset should only contain one specific pest class as objects.

As a reference point, a supervised YOLO model is trained with manual labels, which should achieve at least 0.9 precision and 0.8 recall per class. Precision is prioritized over recall for two reasons. First, recall can be improved later through denser placement of

Authors (Year)	Greenhouse	YOLO	Training
Li et al. 2021 [12]	yes	no	supervised
Rustia et al. 2021 [11]	yes	no	supervised
Rustia et al. 2021 [17]	yes	no	SSL (add. to supervised)
Costa et al. 2021 [13]	yes	yes	supervised
Zhang et al. 2023 [14]	yes	yes	supervised
Li et al. 2024 [18]	no	no	SSL (teacher-student)
Majewski et al. 2024 [19]	no	yes	SSL (pseudo-labels)
Wang et al. 2024 [15]	yes	yes	supervised
Zhou et al. 2024 [20]	no	no	SSL(teacher-student)
Gomez-Zamanillo et al. 2025 [22]	no	no	supervised, add. image processing
Shi et al. 2025 [16]	yes	yes	supervised
Zhou and Shi 2025 [21]	yes	no	SSL(teacher-student)

Table 1: Overview over relevant papers. Colors indicate overlaps to my study with green: identical, orange: overlap, and red: different; yes/no indicate if the respective paper is settled in the greenhouse context or uses a YOLO-model respectively. None of these studies uses a basic self-training approach and none tries any type of SSL with a YOLO-model in the greenhouse setting.

sticky traps, whereas precision depends primarily on model quality. Second, low precision risks erroneous pesticide use, which conflicts with the principles of IPM.

The envisioned adapted automated labeling and SSL is done in two stages. First, bounding boxes for the pest classes are created with segmentation algorithms. Since the images are presorted to contain only one pest class, the detected bounding boxes can be directly associated with the corresponding label. In the second stage, the created datasets are used for self-training a YOLO model. A more detailed overview over the procedure follows in the next section.

2 Methods

2.1 Overview

Main goal of the experiments is to compare a supervised model to a model trained without manual labels. After cleaning and splitting the data (see Section 2.2) a evaluation pipeline is setup to test and compare the models (see Section 2.3). Supervised training was done on tiled and full images and the best model was chosen as baseline (see Section 2.4). Image processing was used to create labels and augmented training data (see Section 2.5) for the alternative approach. These were then used to train models, again both on tiled and full images (see Section 2.6). To improve the model further, false positive prediction were selected as pseudo-labels for retraining the model. Finally the best model was compared to the baseline.

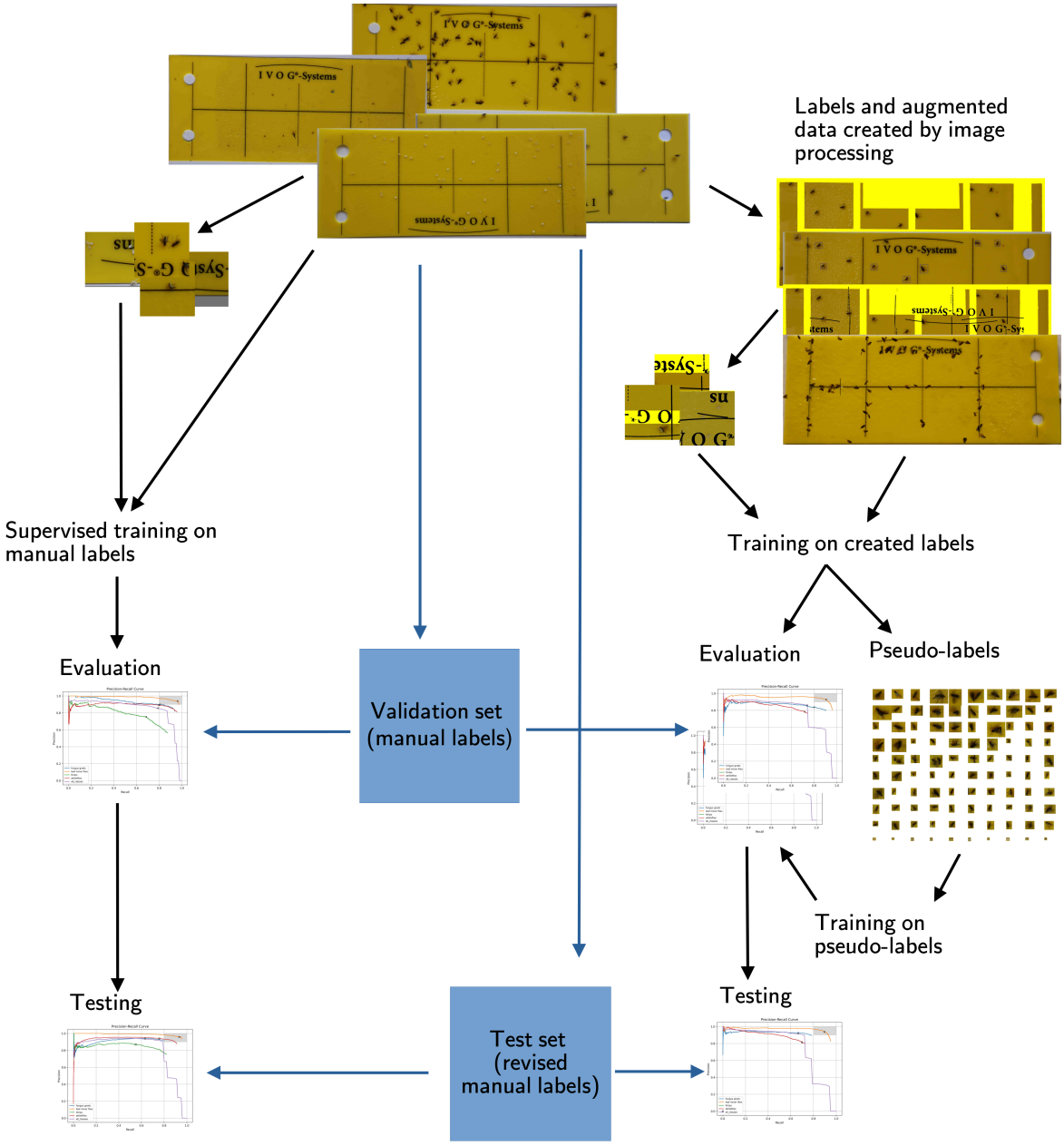


Figure 1: Overview over the experimental setup.

Code and results can be accessed here: https://github.com/ChristophWald/insect_pest_detection.

Common Name	Scientific Name	EPPO Code
Fungus gnats	Bradysia impatiens, JOHANNSEN 1912 (Diptera: Sciaroidea)	BRAIIM
Tomato leaf miner	Liriomyza bryoniae, KALTENBACH 1858 (Diptera: Agromyzidae)	LIRIBO
Western flower thrips	Frankliniella occidentalis, PERGANDE 1895 (Thysanoptera: Thripidae)	FRANOC
Greenhouse whitefly	Trialeurodes vaporariorum, WESTWOOD 1856 (Hemiptera: Aleyrodidae)	TRIAVA

Table 2: Overview of insect pest classes present in the dataset.

2.2 Data

2.2.1 Dataset description

The data is provided by the Julius-Kühn-Institut (JKI) and was collected as part of the Smart-Checkpots project [24] where it was labeled manually and used to train a supervised model. The images are divided into four subsets each associated with only one of four insect pest classes (see Table 2) to simplify the task of manual labeling. Every image in a subset is supposed to depict several instances of only one pest class trapped on a yellow sticky trap (YST) with black background structures. Finding the optimal colors and contrast to attract specific pest classes is a topic of ongoing research (see e.g. [25]). Yellow/black was chosen for this dataset, because it is a compromise for the considered pests and also an industry standard.

The chosen insect pests are common in greenhouses, particularly on crops such as tomatoes and cucumbers, where they can cause severe damage, up to complete crop loss [11]. As Figure 2 shows, the size of the pest classes and thereby the number of pixels they cover differs considerable. In thrips, also male and female differ in appearance. The female individuals are larger and have good contrast against the yellow background, whereas the smaller and more transparent males are harder to detect (compare e.g. row 3, box 2 and 3).

The populated YSTs in this dataset were created at the JKI by breeding the insects to the adult stage in a closed environment (see Figure 3a). For the fungus gnats and the leaf miner flies the YSTs were exposed in the environment for some time until it became populated. For the whiteflies and the thrips the tapping method was used, where the YST is tapped against the plant to catch the insects. After a batch of YSTs was populated,

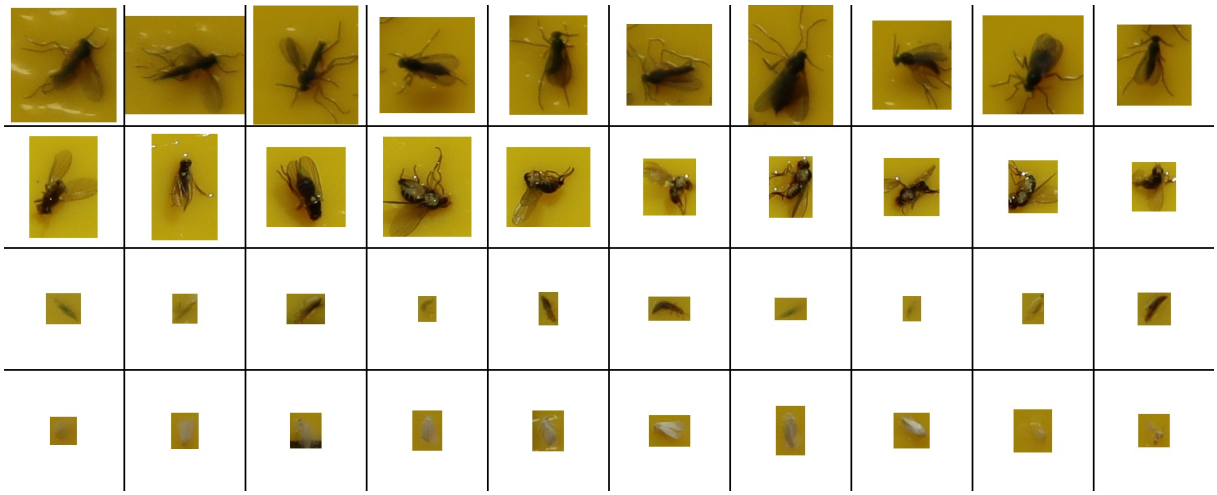


Figure 2: Cropped boxes with fungus gnats, leaf miner flies, thrips, and whiteflies (top to bottom). Original resolution to compare sizes.

(a) Breeding station

(b) Camera setup

Figure 3: Data creation, pictures with kind permission M. Pollmann, JKI

pictures were taken immediately with a Canon EOS M50 Mark II on a tripod at a distance of approximately 13 cm (see Figure 3b) and a resolution of 6000 × 3368 pixels, resulting in files of about 5–7 MB. No artificial light was used to avoid reflections. The pictures have slightly different view angles and different lighting conditions, some still affected from reflections and some having areas with low sharpness.

While the YSTs were captured, few insects could leave the trap and were then trapped by another trap next to it. Therefore the subsets can contain some insects belonging to other subclasses. The tapping method for the whitefly also catched some parts of insects and some foliage of the tobacco plants. Thrips were bred on bean plants which left considerable amount of foliage on the YSTs. Also, thrips in larval stages are trapped,

Class	Images	Labeled	Fraction labeled	Instances	Mean instances per image
Fungus gnats	1328	1018	76.66 %	33373	32.78
Leaf miner flies	1422	659	46.34 %	6518	9.89
Thrips	1134	392	34.57 %	5453	13.91
Whiteflies	1105	433	39.19 %	25735	59.43
Total	**4989**	**2502**	**50.15 %**	**71527**	**28.59**

Table 3: Annotation statistics after cleaning and preprocessing.

which are hard to distinguish from male individuals. In the use case, larvae will cannot be trapped, because only adult can fly.

For each pest class a proportion of images were manually labeled by several persons. Visual inspection showed inconsistency in the correctness of labels, the completeness of labels and the size of the bounding boxes.

2.2.2 Preprocessing

To raise the quality of the dataset, images and labels were checked and improved where possible.

Image files were sorted by their creation time given by the metadata and then the first 100 images for each class were visually inspected, which contained several images with identical YSTs and different camera setting as leftovers of early experiments with the camera setup. These were removed manually. I removed further duplicates (identical images) by comparing file checksums. Images with labels that indicated no insects or unexpected insects not belonging to the class of the subset were also removed.

For the thrips, I inspected the complete subset before splitting the dataset. This led to the exclusion of 200 images with wrong labels or too much other objects. To improve the quality of the thrips labels further I gave the cut out bounding boxes to an expert from the JKI, which identified 1276 labels as incorrect. These were removed. Table 3 shows the number of images and labels after the cleaning reported. The dataset is imbalanced, both in terms of the number of labeled images per class and the mean number of instances per image.

Because of the reported inconsistencies in the labeling, the test set comprising 289 images was checked and corrected by an employee of the JKI, which took about x hours. Unfortunately time constraints prevented a relabeling of the training set, such that supervised training and all validations had to be carried out on a label set of lower quality. The

Class	Kept labels	Added labels	Deleted labels
Fungus gnats	1487	549	92
Leaf miner flies	1241	59	18
Thrips	1113	425	65
Whiteflies	2520	194	102
Total	**6361**	**1227**	**277**

Table 4: Revised label counts for the test set compared to the initial annotations. Leaf miner flies show the smallest changes, while thrips and fungus gnats had a large proportion of previously unlabeled objects. For comparing the evaluation procedure detailed below were used.

results of the supervised training therefore provides only a lower bound for the expected quality of a model.

The summed difficulties with the thrips images and labels led to an exclusion of the class for all experiments but the initial supervised training, which showed that no satisfying results are to be expected in the proposed self-training. There are three major reasons for this decision: low sample size, presence of non-target objects, and low label quality. Because of size and morphological differences between adult males and females the thrips is the hardest pest class to detect and has to be represented with an according number of images and instances in the dataset. Unfortunately, after cleaning the thrips subset has the lowest number of images and labels in the dataset. Furthermore the thrips are easiest to be mixed up with other objects like foliage and therefore require clean images. But the thrips images are the only pest class that has lots of non-target objects, including thrips larvae on their images. The 1276 false labels detected in the inspection of the cut out boxes and the 65 additionally identified false labels in following revision of the test set clearly show that the automated labeling cannot work under the assumption that all objects on an image are instances of the respective pest classes. Finally, the inconsistent labeling quality prohibits proper testing.

Some labels were found to contain negative coordinates. After visual inspection of samples, these values were interpreted as their absolute counterparts, since this matched the actual object positions. The underlying cause of the negative coordinates could not be identified but seems to be connected to the export of labels from Label Studio.

2.2.3 Splitting

Because the number of insects per images is different for the pest classes (see Table 3) sets were balanced where possible. From the labeled images, 200 images for the fungus gnats and the white flies were drawn randomly. For the other classes all labeled images

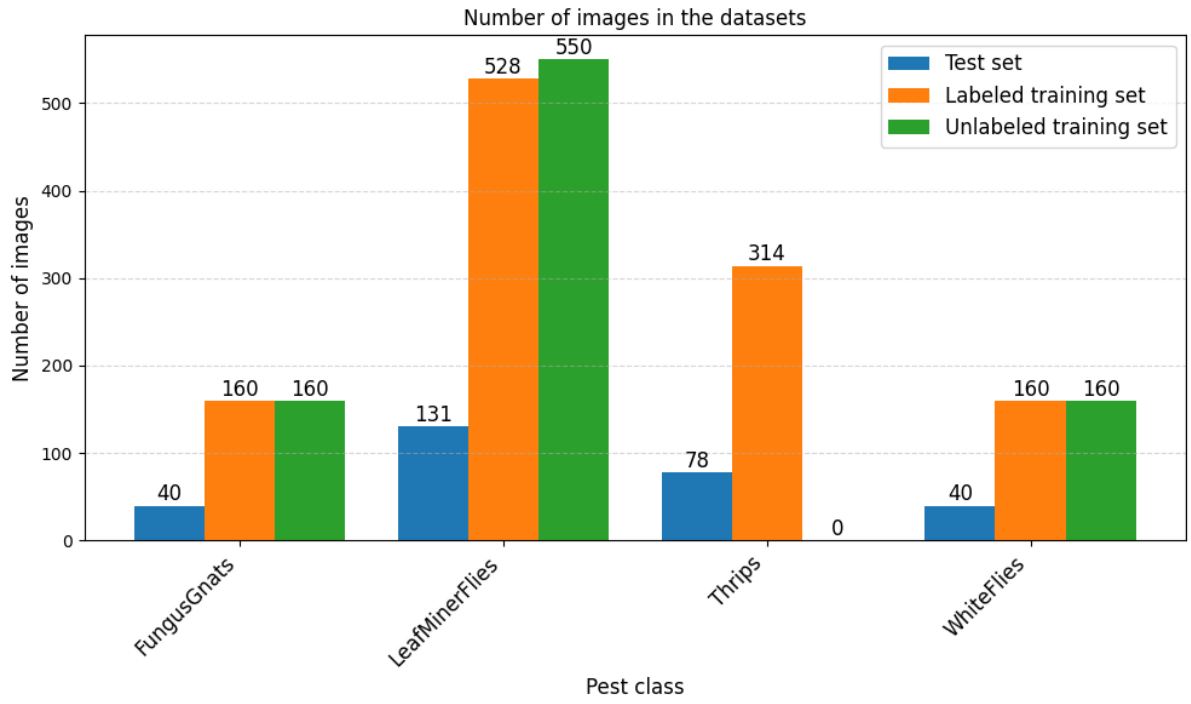


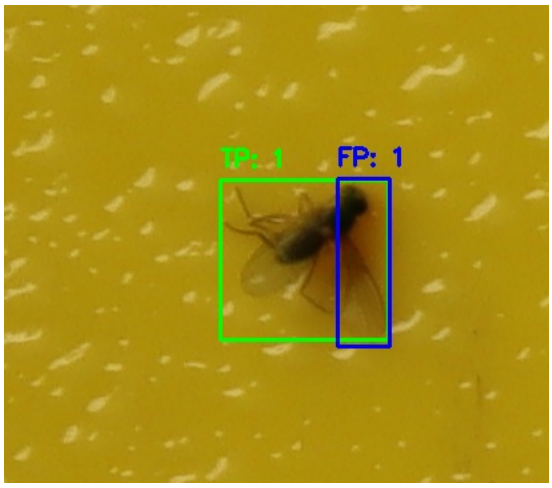
Figure 4: Number of images in the randomly created subsets. Because the absolute number of species per images is different for each class, the number of included images varies.

were used. The labeled set was split into 20% test set and 80% training set, and the training set was further divided into a training and validation set with the same ratio. From the remaining images an unlabeled set with comparable size was drawn to replicate the results of the image-processing and self-training.

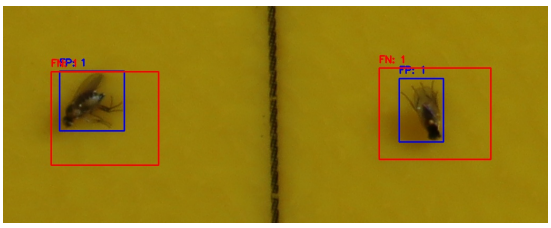
2.3 Evaluation

The evaluation pipeline consists of collecting the predictions of one model, comparing them to the ground truth, finding the best operating points per class and finally comparing the models to each other. Results of parameter tuning (IoU thresholds, stride values, and filter thresholds) are summarized in Table 9. They were tested on two early models trained on tiles with confidence threshold set to 0.5 for all classes, non-maximum supression threshold for filtering before the ground truth to 0.4 and IoU threshold for comparision to ground truth to 0.5 in all tests.

The used YOLO-model only allows prediction on full images and also only allows one confidence threshold for all classes. I used a wrapper script to enable prediction on tiled images and prediction with class specific thresholds. If the prediction is on tiles, a sliding window with a set stride collects the predictions for one image. These are then filtered by



(a) Overlapping bounding boxes not detected for a leaf miner fly not detected by non-maximum suppression (NMS), which creates a false positive box.



(b) Bounding boxes for leaf miner flies counted as false positive (FP) because of low intersection over union (IoU) with the larger ground truth box.

Figure 5: Detection errors caused by low Intersection over Union (IoU) with small boxes. Green boxes are true positives (TP), blue boxes false positives (FP), red boxes false negatives (FN).

a non-maximum suppression (NMS) filter and another filter to exclude contained boxes. NMS relies on the intersection of union (IoU) metric, which is the ratio of the area of two boxes intersected and the area of the union. When two boxes have an IoU above a threshold, only the box with the higher confidence is kept. However, if a small box is completely or partially inside a larger box, the IoU will be low, because the intersection is only as small as the small box and the union is as big as the large box. As visual inspection shows, the small objects in the dataset make it necessary to include an extra filtering for this case. See Figure 5 a for false positives that occur without the extra filter. IoU for NMS was set to 0.4 and the threshold to filter out the other case was set to 0.5. For the stride the values 420 and 440 were tested.

The filtered predictions are then sorted by confidence and are attributed to possible ground truth boxes with greedy matching. Attribution is again defined by IoU and another containment filter, that is implemented, because some manual labels are large, such that much smaller but correct predictions are contained in the ground truth box and produce a low IoU value. See Figure 5 b for false positives that occur without the extra comparision. This evaluation logic was also used for comparing the old and new labels for the test set and for evaluating the final results of the image processing. For IoU a threshold of 0.5 was set, for the extra filter thresholds between 0.5 and 0.9 were tested.

For defining the quality of a model, per class precision-recall curves are built and confidence thresholds that are closest to the target zone of precision greater or equal than 0.9

and recall greater or equal 0.8 are taken as operating points for the model. If the curve reached the target zone, the point with the best F1 value inside the zone was chosen. Different models are then compared by the number of operating points inside the target zone and the distance of the remaining operating points to the target zone.

2.4 Supervised training

As baseline for the automated labeling process, a YOLOv8 model is trained using the manual labels provided. The aim is to ascertain that the data is sufficient to reach the target metrics of a minimum per-class precision of 0.9 and recall of 0.8. Since supervised training is not the main focus of this project, experiments are not designed to fully optimize results. A small YOLOv8s model (released 2023 by Ultralytics, 11.2 million parameters at 640-pixel images [26]) pretrained on COCO is chosen because initial tests showed that larger models or later versions did not considerable improve performance and because the codebase of later models is harder to adapt. No task-specific pretrained models are available to my knowledge.

The images are cropped to the size of the YST. This decreases the size of the dataset by 17.1 GB but also introduces different image sizes. Because YOLO-formatted labels are relative to the image size, these have to be recalculated.

Training YOLO with the given image sizes needs exceedingly large amounts of GPU RAM. Therefore in the training with full images, these were resized to 1280 pixel. As an alternative, the images were tiled into 640x640 patches for training. For this, the images were padded and then tiled with stride 440. The overlap of 200 pixels corresponds to the size of the largest pest class, the fungus gnats, such that an instance is at least contained in one tile. The labels are recalculated according to the tiles and are only kept if at least 80% of the bounding boxes lie inside the respective tile. The resulting tiles contain a large number of empty background tiles, which account for 68% in the labeled training set.

Starting from the default settings, short training runs with selected parameter changes were conducted to identify optimization potential, where augmentation settings and the ratio of background to foreground images are identified as the main factors influencing performance. Changing the default optimizer (stochastic gradient descent), learning rate schedule, and loss weights did not yield improvements. Because the dataset already contains considerable variance (see Section 2.2.1), augmentations are applied only subtly. The possible scaling factor is reduced to 30% to avoid unrealistic size differences between objects, and the probability of mosaic augmentation (introduced in 2020 with YOLOv4 [27]) is lowered to 25% to prevent the model from learning lighting differences between

Training	Dataset	Background ratio	Epochs trained	Best epoch
S1	tiles	68%	200	144
S2	tiles	30%	200	127
S3	tiles without thrips	47%	200	136
S4	full images	0 %	100	94

Table 5: Selected training runs for supervised training

images. Horizontal flipping was disabled as the images already had mixed orientation. A small amount of mixup ([28]) is introduced with a 5% probability. All other parameters are kept at the default values, which include jitter for all channels of the HSV-encoded image, translation, vertical flips, RandAugment ([29]) and random erasing ([30]).

All models were trained up to 200 epochs, then the best epoch (measured by the mAP) was taken.

2.5 Image processing

The aim of the image processing is to detect a proportion of objects and create labels including bounding box coordinates, which will serve as the starting point for the training of a YOLO model. The main idea of this step is to separate background and foreground with a color threshold and then keep all foreground contours as target objects. A rectangle around the contour serves as bounding box and the class id can be derived from the subset of the image, as all insects on an image are of the same pest class. For images without background structures the process would reduce to the short procedure reported in section 2.5.2. However, when images contain background structures—as in the dataset used in this study—more complex preprocessing is required to mask out those structures, as detailed in Section 2.5.1 and augmentation is needed to train the model on occurrences of the target objects in combination with the reintroduced background structures, which is given account on in Section 2.5.3

The presented pipeline was developed with the labeled training set with raising the absolute number of true positives and minimizing the number of false positives as main objectives. Precision is also tracked but considered secondary, as it might come at the cost of too little true positives. Binarization, which is used at several steps, is achieved through thresholding in the HSV color space, because the hue component provides robustness against variations in lighting. The thresholds used were: Hue 20–30, Saturation 100–255, and Value 100–255.

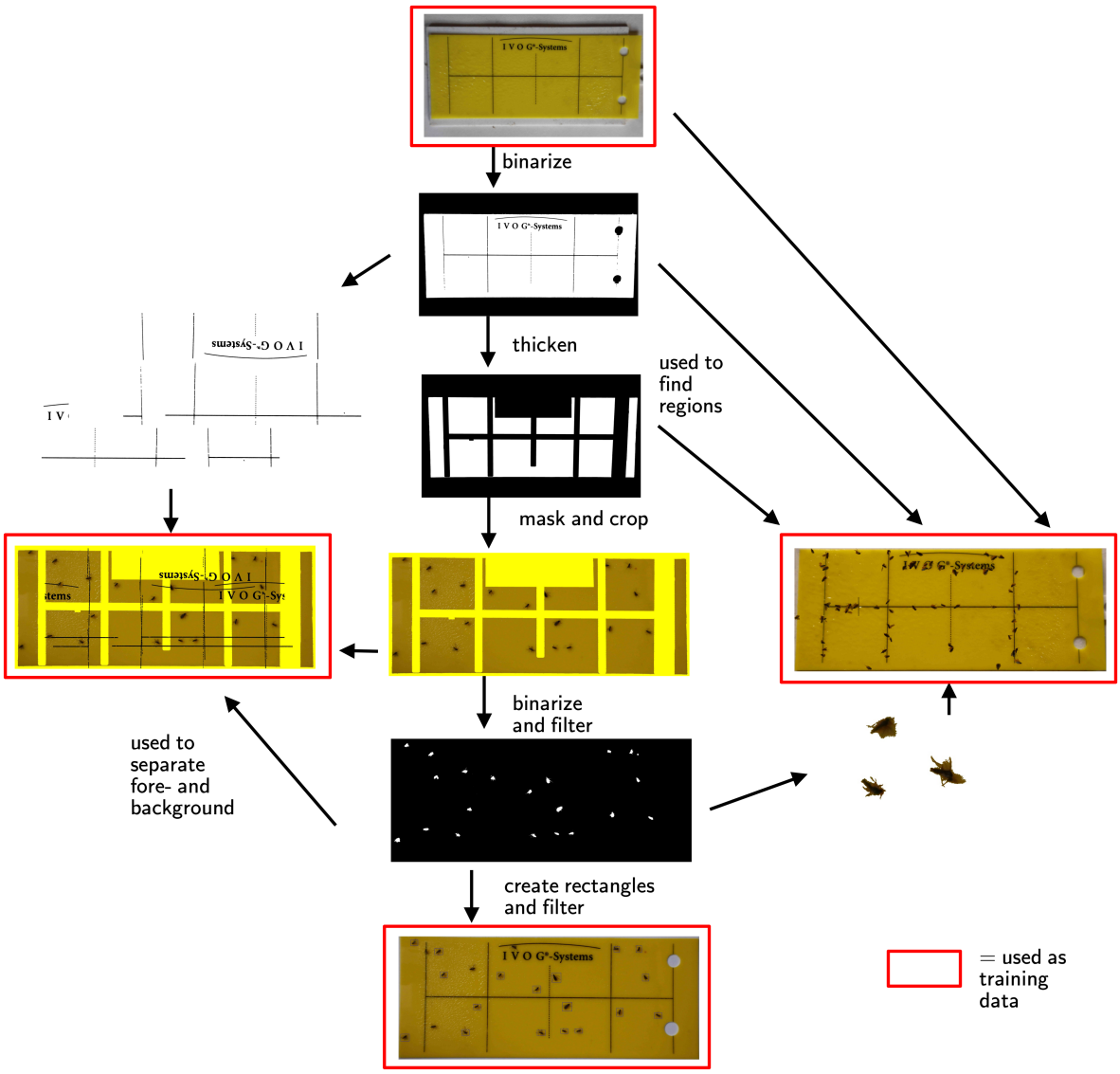
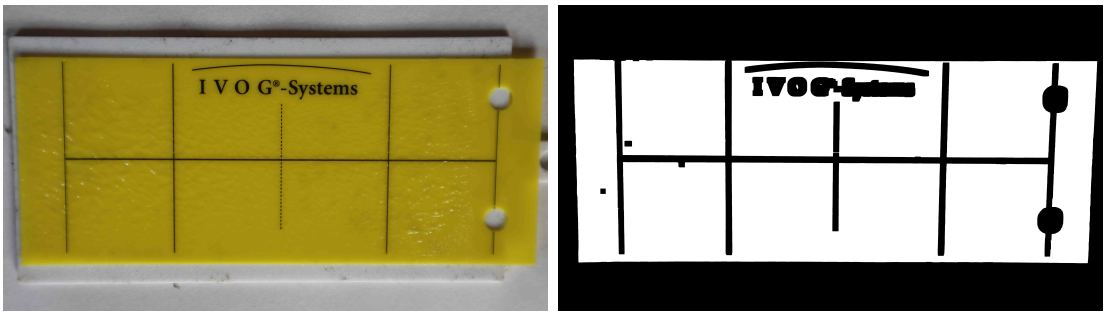


Figure 6: Overview of creating training data with image processing.

2.5.1 Preprocessing for segmentation

To filter out background structures, a mask covering these regions is applied to each image. The pixels within the mask are changed to a yellow hue, allowing them to be identified as background during segmentation. Direct detection of the lines was tested, but led to no usable results, as the detected line segments were often very short and could not be separated from the objects. Because the images in the dataset were captured from slightly different angles, the mask must be aligned individually for each image. All image processing is performed using the cv2-Python library.

Since the masking procedure assumes the header is located at the top of the YST, images were rotated when necessary and labels adapted. The required rotation was detected



(a) Image used for the creation of masks.

(b) Mask created by overlaying several images and then thickening the lines.

(c) Mask from b) with thicker lines.

(d) Mask from c) with a rectangle as header and the holes covered.

Figure 7: Masks tested for masking out the background structures.

by dividing the YST area into horizontal halves and comparing the summed black pixel amount in each half, with black pixels defined as having a value below 50. This method becomes unreliable when the YST is crowded with a large number of black insects, so a final visual inspection was performed to ensure correctness.

Creating the mask required balancing line thickness to fully cover background structures despite minor misalignment, while avoiding excessive coverage of the objects. Binarized and denoised images of empty YSTs, captured under the same conditions as the dataset images, were used for mask generation. The masks were created by combining several empty YST images with projective transformations and then thickening the black structures with dilation (kernel size 25) (Figure 7b). From this mask, two additional variations were generated by applying further dilation (kernel size 50) (Figure 7c) and by manually thickening the right border lines to cover the holes completely and drawing a rectangle over the header (Figure 7d). For subsequent alignment, the corners of the YST contour and two points of the horizontal midline calculated from the original masks were saved.

With the masks prepared, preprocessed and masked versions of the image set were created. The pipeline of this process is divided into the following steps:

- Find contours on the binarized image. Take the largest, which is the YST (algorithm used: cv2.findContours based on [31]).

- Assuming a rectangle shape find the corners (algorithms used: cv2.arcLength and cv2.approxPolyDP based on the Douglas-Peucker algorithm [32]).
- Using the corners of the mask and the image, do a projective transformation of the mask and of the saved midline points of the mask (algorithms used: cv2.findHomography based on RANSAC [33], cv2.warpPerspective).
- Using the midpoints of middle horizontal lines of the mask and the image, do another affine transformation of the mask: (algorithms used: cv2.HoughLines based on the generalized Hough transform [34], cv2.perspectiveTransform, cv2.warpAffine)
- Color the pixels given by the aligned mask in the image yellow to mark it as background.
- Crop the image to the YST (and recalculate the manual labels for evaluation).

For about 6% of the images, the corners of the YST could not be detected. These images were dropped.

2.5.2 Segmentation

After masking, the images can be binarized with the defined thresholds and the foreground objects can be segmented. The results still need to be filtered to exclude unwanted objects. A first set of thresholds is used to evaluate the effects of the different masks. A second, more refined set of thresholds is derived from inspecting the data given by the best-evaluated mask (Figure 7d). Evaluation was done on the manual labels of the labeled training set. Unlike in the evaluation of model predictions, the comparison of the bounding boxes was reduced to a simple intersection between rectangles with a threshold of 0.5, because it gives nearly identical results for smaller predictions contained in the ground truth and is more strict for overlapping ones.

In the first filter set for comparing the masks, lower and upper thresholds for the detected contour areas were derived from visual inspection (Table 6). The rectangles around the contours are then scaled with a factor of 1.5, because the contours lose small features like legs, which would otherwise make the rectangles too small. Then another threshold was applied to the rectangles. Assuming that more oblong rectangles are likely to be non-insect objects (e.g., leftover background elements, foliage, or parts of insects), the maximum ratio of height to width was set to 2. This excludes rectangles with one side being more than twice as long as the other side.

Species	Threshold	Filter set 1	Filter set 2
Fungus gnats	Min contour	1000	*same*
	Max contour	10000	*same*
	Max box size	-	95th percentile
	Max box ratio	2	95th percentile
Leaf miner flies	Min contour	1000	*same*
	Max contour	10000	*same*
	Max box size	-	95th percentile
	Max box ratio	2	95th percentile
Whiteflies	Min contour	100	*same*
	Max contour	1000	2000
	Max box size	-	95th percentile
	Max box ratio	2	95th percentile

Table 6: Thresholds of the two filter sets for filtering the results of the segmentation.

The results from the best-performing mask (Figure 7d) are used to inspect the distribution of box sizes and box ratios. Additional filter thresholds were then tested on the calculated rectangles and are given in Table 6. The goals were to reduce the number of small insect parts and occasional non-insect objects, as well as the number of large boxes with more than one insect occluding each other. Only for the whiteflies did overlapping boxes pose a problem, presumably because individual objects often led to two separate contours. Therefore this case was checked manually and only one rectangle was kept. To adapt to the variations in saturation and value between individual whiteflies, a more intricate binarization as well as additional value thresholding was tested unsuccessfully.

2.5.3 Augmenting the data

Masking the background structures excludes all insects on lines or letters and creates a biased label set. A model trained on this dataset will inherit this bias and learn to ignore insects on background structures. To prevent this, two types of augmentation sets combining insects and the structures were created. The first set distributes background structures randomly on the masked images. The second set starts with images of empty sticky traps and distributes cut-out insects on and near the background structures (see Figure 6 for examples).

Specifically, the first augmentation set was created by combining three layers: the background of a masked image, a middle layer containing the prepared background structures from images of empty YSTs, and the foreground layer of the masked image, which contains the insects. The middle layer itself consists of two layers of background structures that were cut out to remove unwanted parts (border lines, holes) and then rotated and shifted to create variations. This procedure was applied to the complete training set.

The second augmentation set uses empty YSTs as the starting point, which were varied in lighting, vertical orientation, and by shearing them. Two different masks were used to define areas on the YST where insects should be placed, ensuring objects are positioned directly on or near the structures. Cut our Insects were then placed randomly within these areas. For each pest class, 24 images were created.

The insects for the second set were extracted from the original images guided by the bounding boxes created by the method described above. The contours of the individual insects were cut out, with binarization using fixed thresholds that were refined as needed. As the detected whitefly contours tend to form loops with empty centers, the pixels inside these loops were identified and also marked as foreground. For fungus gnats and leaf miner flies, individually calculated value thresholds were obtained using Otsu's method [35]. Additionally, contours found inside each bounding box were filtered to retain only the largest contour, excluding artifacts. To address a second source of bias in the created labels, value variations were also introduced for the whiteflies. The magnitude of these variations was derived from inspecting the value distribution of true positives and false negatives for the whiteflies, which showed a clear bimodality.

2.6 Self-Training

A model was trained on augmented and unaugmented images with the labels created by image processing. In a second step the model was used to predict on the unaugmented, original images to create pseudo labels for the insects missed by the masked-based label creation. This second step was then repeated to further improve label quality.

2.6.1 Training on the labels created by image processing

The model used the same setting as explicated for the supervised training, but as the images were all oriented upwards in the masking process, additional random vertical flips were tested. Learning rate (0.00125) and Optimizer (AdamW) were used as set by YOLO. For validation, the manual labels were used. Main variable for the experiments was the composition of the training data tested on tiled data. Starting with the labeled original image, the addition of empty images and the augmented set 2, but also the replacement of the original images with the augmented set 1 as basic training data was tested. In final tests, the best composition was also tested on full images. The results were also replicated with the unlabeled training set.

Run	Flipud	Epochs	Basic Data	Empty Tiles	Augmented Set 2
1	no	10	tiles	no	no
2	no	10	tiles	yes	no
3	yes	20	tiles	yes	no
4	yes	20	tiles	yes	yes
5	yes	20	augmented tiles 1	yes	no
6	yes	20	augmented tiles 1	yes	yes
7	yes	20	augmented 1 full images	no	yes

Table 7: Training runs for training on the image processed labels. Flipud turns the image 180° with probability 50%. 'Tiles' denotes the unaugmented dataset and include only tiles with at least one label. Rescaling for full images was set 1280 pixels as in supervised training.

2.6.2 Further training on predicted labels

After the first training on the created labels, the model could distinguish classes and also separate foreground and background objects. To improve the model, its capability to create false positives that can be used as pseudo-labels for further training was tested. Using a model to predict labels that is then trained on again is the simplest method of the self-training paradigm, originally introduced as "pseudo-labeling" by Lee (2013) [36].

Pseudo-labels have to be filtered to exclude actual false positives from not yet labeled target objects. The thresholds tested were derived from visual inspection. The main trade-off is allowing enough pseudo-labels for improving the model without degrading its predictions by the inclusion of wrong objects.

The first iteration transfers the rules learned on the augmented data to the original images with the background structures included. False positives consist mainly of insects on the background structures and the threshold has to be low because otherwise many insect on lines or letters remain unlabeled and data contains contradicting information. Further iterations are used to enhance the quality of the predictions by using higher threshold adapted to each class. Train length is set to 20 epochs for all runs.

Several unsuccessful experiments are not reported; these included shorter training runs combined with higher confidence thresholds for the initial pseudo-labels. Adding new images to the training set, which were then completely pseudo-labeled, was also tested, as well as implementing a class-loss weighting based on the confidence of the predicted pseudo-label.

Run	data	thresh fungus gnats	thresh leaf miner flies	thresh whiteflies
6.1	tiles	0.25	0.25	0.25
6.2	tiles	0.8	0.8	0.7
6.3	tiles	0.7	0.7	0.6
7.1	full images	0.25	0.25	0.25
7.2	full images	0.5	0.8	0.7

Table 8: Training runs for the self-training with additional pseudo-labels filtered by class-specific confidence thresholds for tiles. The first number refers to the first training of the model given in Table 7, second number gives the iteration. Flipud probability was always set to 50%, training was always 20 epochs.

3 Results

3.1 Comparing supervised training and self-training

Overall, training on full images produced better results. The best model is the supervised model on the manual labels (S4). In validation it arrived the target zone for all but the incoherently labeled thrips. Because confidence thresholds are derived from the incompletely labeled validation set, they did not provide the thresholds with the best performance for the model on the test set, where it missed the target zone for all but the leaf miner flies. But as can be seen in the precision-recall curve on the test set the model increases precision overall in the test set, because the more complete labels lead to lower false positive rates.

The best self-trained model (7.2) arrived the target zone only for the leaf miner flies, although it improved on the image processed labels.

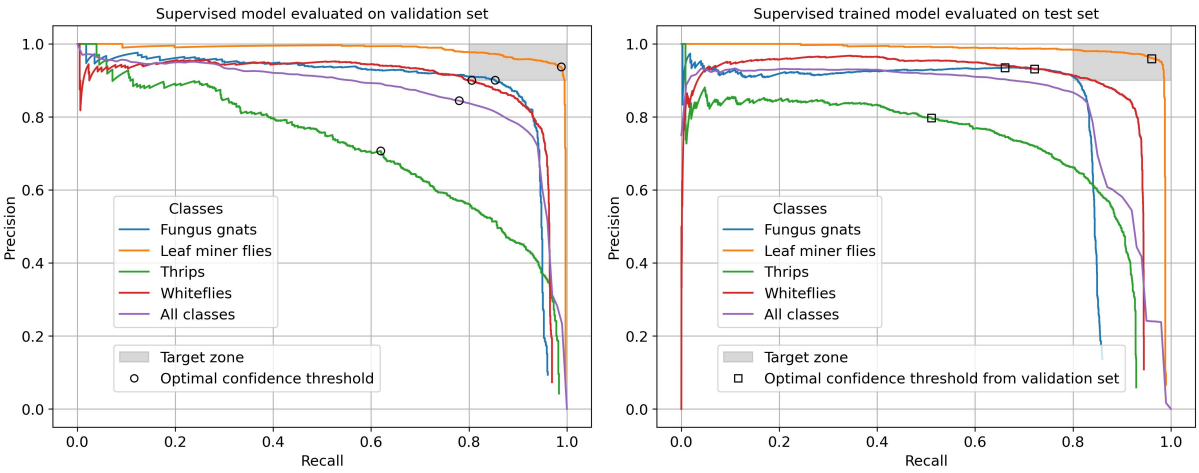


Figure 8: Precision-recall curves for the best model trained supervised on full images (S4) on the validation set (left) and test set (right) with the optimal thresholds derived from the validation set marked in both. Training the model on the incomplete labels would still provide a sufficient model for all but the thrips, but deriving the confidence thresholds from them makes the model miss its best performance on the completely labeled test set.

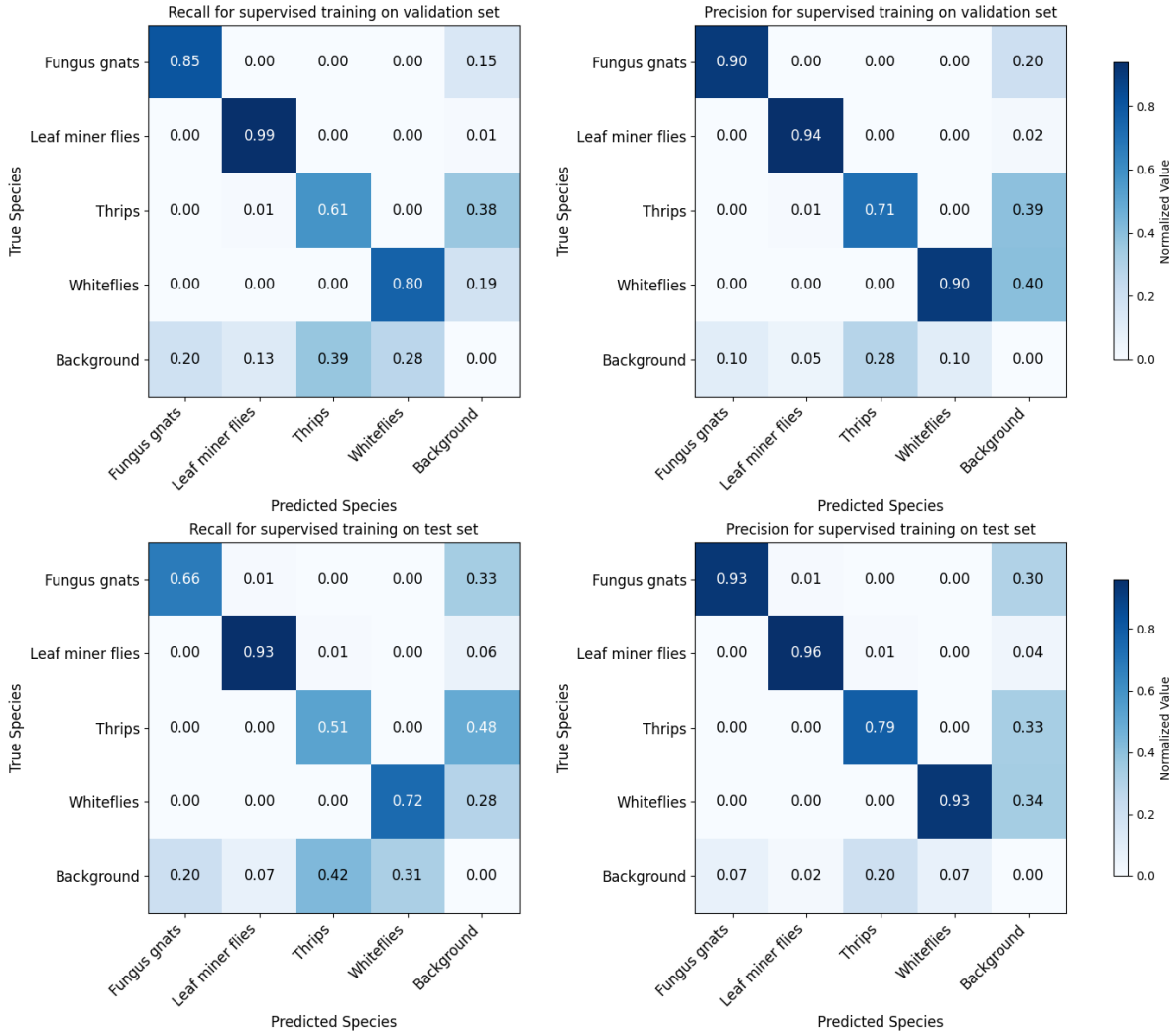


Figure 9: Confusion matrices for the supervised training on the validation set and the test set. The thresholds derived from the validation set lead to higher precision and lower recalls in the test set.

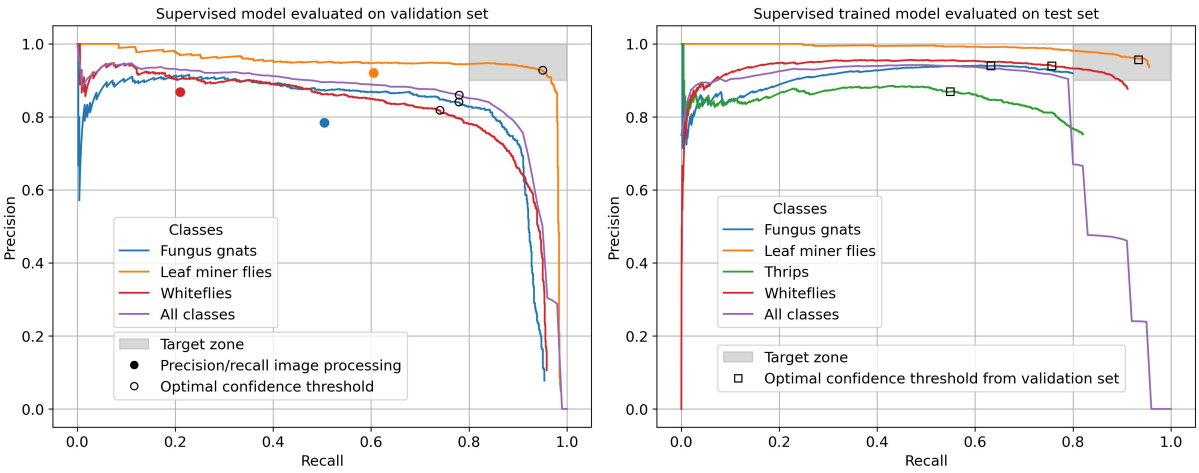


Figure 10: Precision-recall curves for the best model self-trained on full images (7.2) on the validation set (left) and the test set (right) with optimal confidence thresholds dervived from validation set marked in both. In the left image, results of images processing are marked as coloured circles, too. Self-training did improve the created labels, but did not provide a sufficient model.

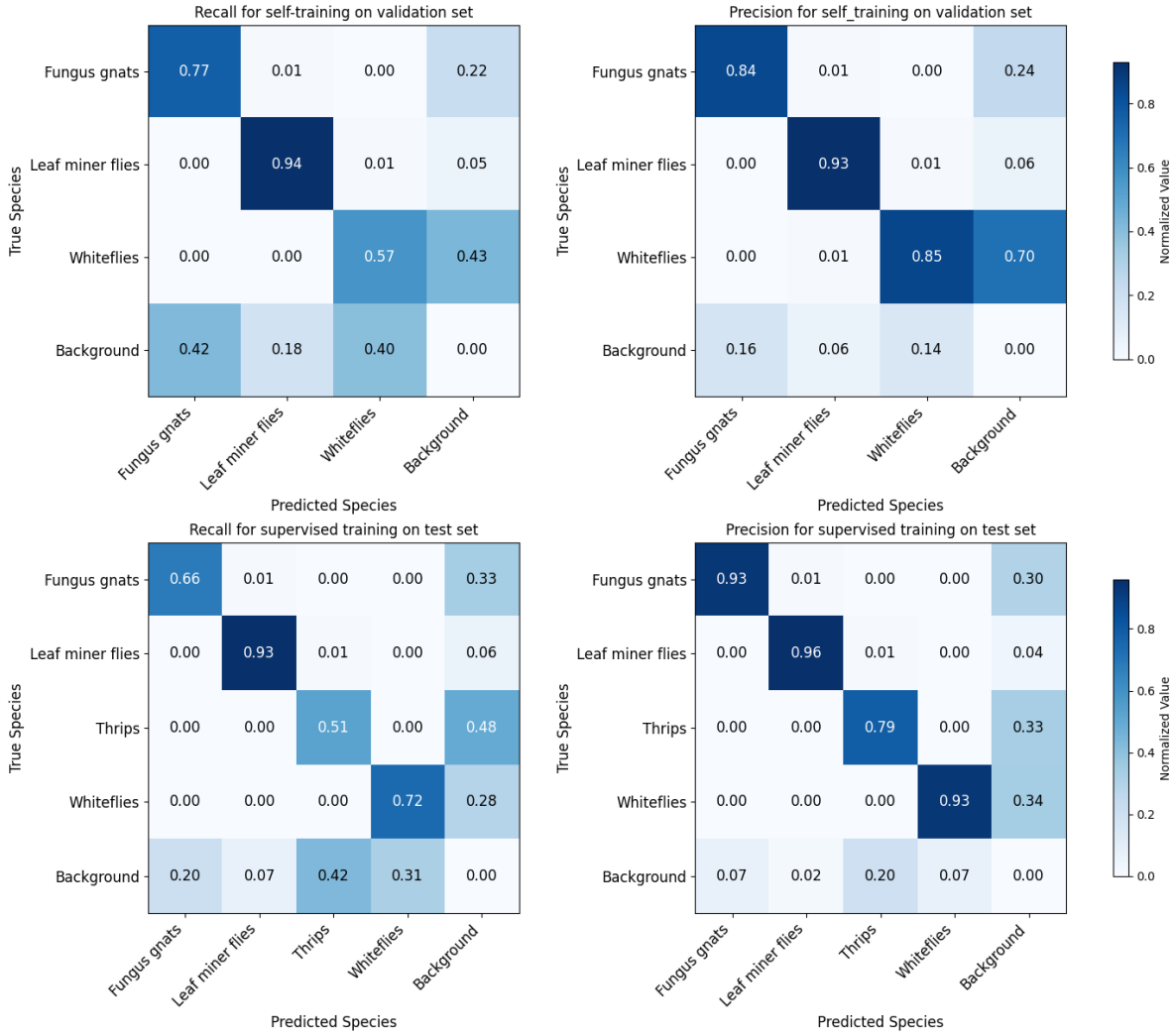


Figure 11: Confusion matrices for the self training on the validation set and the test set. Again the thresholds derived from the validation set lead to higher precision and lower recalls in the test set.

Stride	Containment filter	Containment comparison	S1			S2		
			TP	FP	FN	TP	FP	FN
440	-	-	3240	637	1024	3469	664	798
440	0.5	-	3240	592	1027	3469	664	798
440	0.5	0.9	3331	501	936	3469	584	798
420	0.5	0.9	3569	540	698	3737	716	539
440	0.5	0.5	3601	508	666	3469	587	798
420	0.5	0.5	3601	508	666	3737	637	530

Table 9: Comparison of detection results under different settings of extra thresholds with given absolute numbers for true positives (TP), false positives (FP) and false negatives FN). The models are trained on tiles, each 100 epochs, S1 on the full trained label set, S2 with only 30% of the empty background tiles. The chosen parameters in the last row improve results for both models.

3.2 Evaluation settings

For all tests threshold 0.5 for filtering predictions contained in larger predicted boxes as well as allowing contained predictions contained inside ground truth boxes were selected and stride 420 for models trained on tiles. These settings never degrade performance, but the scale of improvement differs on different models. Tests with the supervised models S1 and S2 showed that changing the stride to 420 always brings a boost for the number of true positives. The extra containment filter before comparing the boxes brings a slight reduction in false negatives only for model S1. The extra containment section in the comparing to the ground truth with 0.9 always brings benefits in all three metrics, but setting the threshold to 0.5 only improves results for stride 420.

3.3 Supervised training

The supervised model on trained on the tiles (S1) was slightly worse then the one trained on the full images (S4).

3.4 Image processing

3.4.1 Preprocessing for segmentation

The best mask (Figure 7d) decreased the number of false positives while retaining a sufficient number of true positives for the self-training (see Figure 13). It was used to produce a set of contours and rectangles with the simple filter set 1 (see Table 6). From

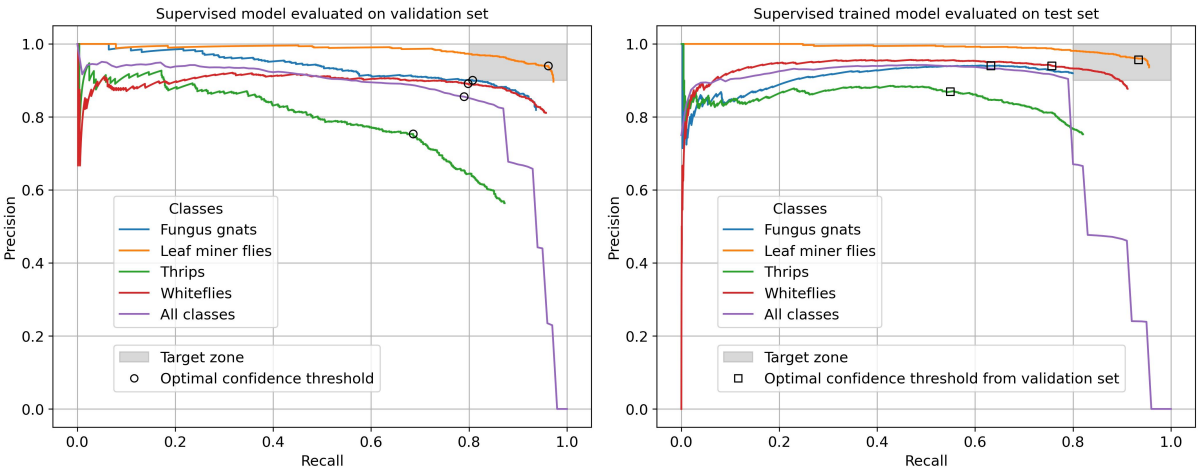


Figure 12: Precision-recall curves for supervised training on tiles. The results are slightly worse than supervised training on full images (see 8) for all but the thrips. Again the model suffers from the confidence thresholds derived from the insufficiently labeled validation set.

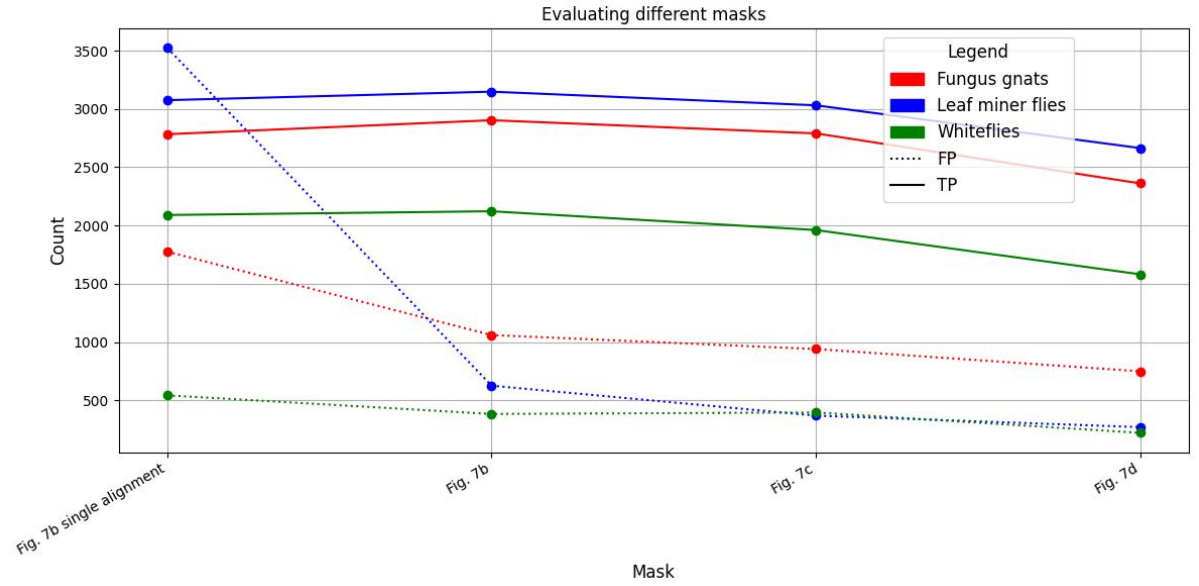


Figure 13: Evaluating the masks given in Figure 7. The second vertical shift added to the the single alignment of the mask to the image is most important for the leaf miner flies as the comparision with the thinner mask 7b shows. In general thicker mask reduce the number of false and true positives.

the distribution of box sizes and side length ratios, filter thresholds for filter set 2 are taken. Table 11 gives the found values for the metrics for both the labeled and the unlabeled training set, which are similar as expected.

Training Set	Metric	Fungus Gnats	Leaf Miner Flies	Whiteflies
Labeled Training Set	95th percentile box size	42260	23544	3275
	95th percentile box ratio	1.74	1.77	1.88
Unlabeled Training Set	95th percentile box size	43456	28480	3477
	95th percentile box ratio	1.73	1.76	1.84

Table 10: Found possible thresholds for labeled and unlabeled training set.

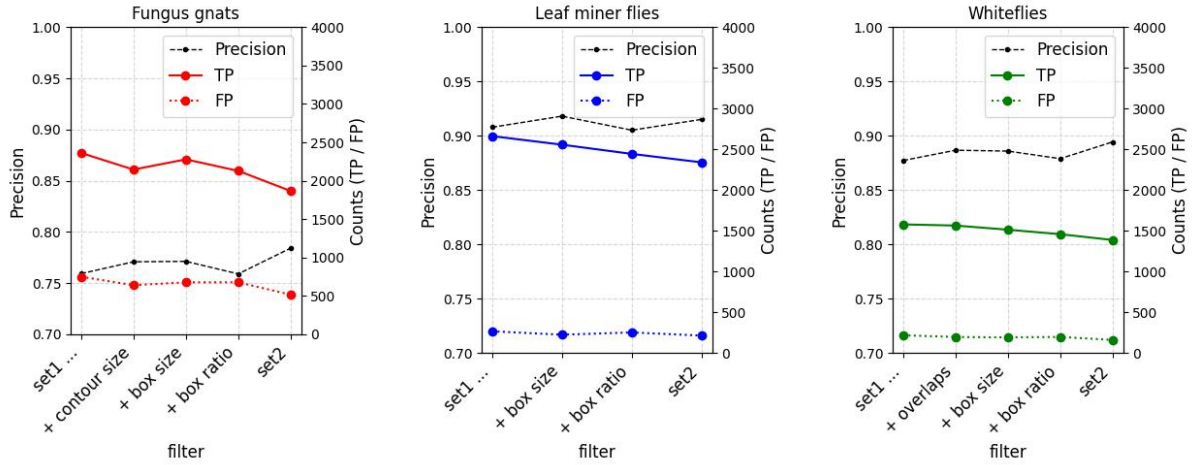


Figure 14: Results for different filters for each species. Leftmost is always the parameter set 1 with no extra filtering, rightmost is parameter set 2 with all changes applied. In between single additions to set 1 are tested.

Class	Labeled training set		Unlabeled training set	
	Filter set 1	Filter set 2	Filter set 1	Filter set 2
Fungus gnats	3110	2380	3477	2699
Leaf miner flies	2934	2684	3116	2628
Whiteflies	1802	1575	5015	4308

Table 11: Absolute number of rectangles created.

3.4.2 Segmentation

Figure 14 shows the results for the tests with the thresholds derived by statistics. For all pest classes precision is increased but also the number of true positives reduced.

As to be expected objects of the same color, shape and size cannot be filtered out. This concerns mainly parts of the insects, as well as insects occluding each other. Table 11 gives the absolute numbers of rectangles returned with both filter sets for the labeled and unlabeled training set. As expected they turn out to be very similar with exception of the total numbers of whiteflies, which is doubled in the unlabeled training set. As the number of filtered out rectangles is also proportionally larger, this is possibly only a effect of more crowded whitefly images in the unlabeled set.

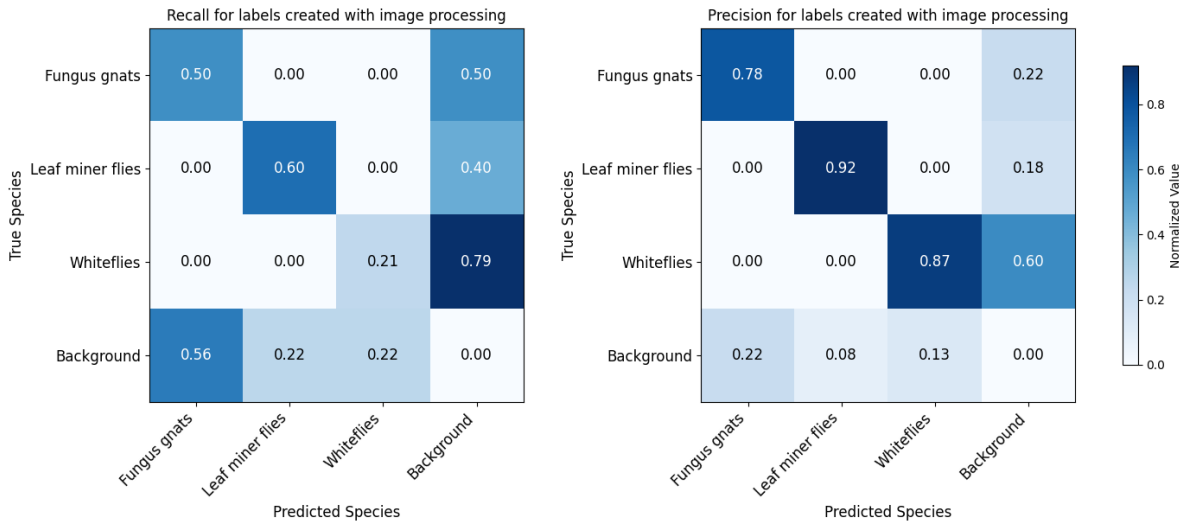


Figure 15: Confidence matrices for the image processed masks. Low recall is mainly due to the insects on the background structures that were masked out. The training improved recall without degrading precision (cf. Fig 11), because it can predcit the insects on background structures.

3.5 Self training

For both tiles and full images, the combination of both augmented data sets 1 & 2 provided the best dataset for training. For tiles the inclusion of empty tiles provided some improvements. As reported above, the training on full images yielded the best results. Compared to the labels created by image processing it improved recall for all classes while retaining precision (compare the confusion matrices 11 and 15). After transferring the labels to the original images in the first iteration, further iterations with new pseudo-labels only brought marginal improvements before stagnating after two (for full images) and three (for tiles) iterations. Visual inspection of the pseudo-labels showed no differentiation of insects and other objects by confidence. Visual inspection of the predictions of the best self-trained model (7.2) on the test set accordingly showed false positives for foliage and insect fragments, but also a larger number of still unlabeled and correctly predicted fungus gnats and whiteflies on some images. Correct labeling will increase the number of true positives and decrease the number of false positives, which in turn will raise precision and also recall.

4 Discussion

The results confirm that the state-of-the-art supervised training approach can provide optimal performance. They also reaffirm its main limitation: the need for a large number of accurate manual labels, which only trained experts can provide consistently.

The alternative approach, using labels created through image processing, works reliably when the images fulfill the necessary requirements, as demonstrated by the sufficient results for the leaf miner flies. After only one short training run on augmented data and a retraining on the labels predicted for the original images, a model can predict with the required precision and recall. In contrast, the results for fungus gnats and whiteflies suffer from the high number of insect fragments. This cannot be separated by image processing, because fragments are similar to complete insects. Because the confidences give no clear distinction between wrong and false objects, confidence thresholding or weighing by confidence in the pseudo-labeling is not a viable solution, either. In the final evaluation on the test precision and recall may be slightly underestimated because of the remaining unlabeled insects present in the test set. However, the rapidly decreasing precision–recall curve may also indicate a yet unaddressed additional bias introduced by the image processing.

The experiments demonstrate that data quality is the primary factor influencing model performance. Improving data quality can be achieved at two stages: during data acquisition or during labeling. Time can be invested in creating cleaner data from the outset, for example by improving the methods used to catch the insects or by cleaning the images manually or digitally. If sufficient effort is invested and resources are available to adapt the image-processing steps to each pest class, the tested approach could probably replace most manual labeling. However, for noisy data image processing is bound to fail, and for supervised training time has to be invested to create sufficient, high-quality labels. In all cases, manually labeled validation and test sets remain necessary, and labeling in general should follow clear guidelines and for difficult objects involve collaborative review to ensure consistent annotations.

Another important aspect of data quality is class balancing. An efficient solution would be to estimate the number of insects caught per image and then calculate the required number of images accordingly. If a pest class presents additional challenges—such as the differing appearances of male and female thrips—the number of images or instances must be adjusted accordingly.

Rather than completely replacing supervised training, the approach tested here could be integrated to make the labeling process more efficient. Labeling software often allows the inclusion of models for pre-labeling, for which the self-trained model presented here would be well-suited, since in many cases labeling would be reduced to simply adding or removing a small number of bounding boxes. For a single project, manual labeling is more efficient than first developing a model to predict labels. But for projects that aim to expand the number of detectable pest classes continuously, a tool to generate pseudo-labels could streamline the inclusion of new classes. Unlike supervised training, where

labeling requires the same amount of time for each new class, the bulk of development costs for such a tool occur only once. This could also help mitigate additional challenges outside the scope of this project, such as transferring trained models to images captured at different resolutions or adapting to other differences between training set and data from the use case, e.g. new sticky traps.

In conclusion, the tested alternative approach is highly dependent on the quality of the data. Lower quality correlates with higher development costs, since each problem requires an individual adaptation. In contrast, the state-of-the-art supervised approach depends primarily on the quality of the provided labels and, with correct labels, produces sufficient results in less time and with comparatively low development costs. Nevertheless, an adapted version of the alternative approach may facilitate the labeling process and could be an option for long-term projects.

5 References

[1] Curtis A. Deutsch et al. "Increase in crop losses to insect pests in a warming climate". In: *Science* 361.6405 (2018), pp. 916–919. DOI: 10.1126/science.aat3466.

[2] Laure Mamy et al. *Impacts des produits phytopharmaceutiques sur la biodiversité et les services écosystémiques. Rapport de l'expertise scientifique collective*. Research Report. INRAE ; IFREMER, 2022, 1408 p. DOI: 10.17180/0gp2-cd65.

[3] Matthew R. Smith et al. "Pollinator Deficits, Food Consumption, and Consequences for Human Health: A Modeling Study". In: *Environmental Health Perspectives* 130.12 (2022), p. 127003. DOI: 10.1289/EHP10947.

[4] URL: https://www.fao.org/pest-and-pesticide-management/ipm/integrated-pest-management/en/ (visited on 04/04/2025).

[5] G.J. Messelink and H.M. Kruidhof. "Advances in pest and disease management in greenhouse cultivation". In: *Achieving sustainable greenhouse cultivation*. Ed. by L. Marcelis and E. Heuvelink. United Kingdom: Burleigh Dodds Science Publishing, Sept. 2019, pp. 311–356. DOI: 10.19103/as.2019.0052.17.

[6] URL: https://food.ec.europa.eu/plants/pesticides/sustainable-use-pesticides/integrated-pest-management-ipm_en (visited on 04/02/2025).

[7] Tiago Domingues, Tomás Brandão, and João C. Ferreira. "Machine Learning for Detection and Prediction of Crop Diseases and Pests: A Comprehensive Survey". In: *Agriculture* 12.9 (2022). DOI: 10.3390/agriculture12091350.

[8] Ana Cláudia Teixeira et al. "A Systematic Review on Automatic Insect Detection Using Deep Learning". In: *Agriculture* 13.3 (2023). DOI: 10.3390/agriculture13030713.

[9] Mamta Mittal et al. “Machine learning for pest detection and infestation prediction: A comprehensive review”. In: *WIREs Data Mining and Knowledge Discovery* 14.5 (2024). DOI: https://doi.org/10.1002/widm.1551.

[10] Joseph Redmon et al. “You only look once: Unified, real-time object detection”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788. DOI: 10.48550/arXiv.1506.02640.

[11] Dan Jeric Arcega Rustia et al. “Automatic greenhouse insect pest detection and recognition based on a cascaded deep learning classification method”. In: *Journal of Applied Entomology* 145.3 (Apr. 2021), pp. 206–222. DOI: 10.1111/jen.12834.

[12] Wenyong Li et al. “Field detection of tiny pests from sticky trap images using deep learning in agricultural greenhouse”. In: *Computers and Electronics in Agriculture* 183 (Apr. 2021), p. 106048. DOI: 10.1016/j.compag.2021.106048.

[13] Dinis Costa et al. “Intelligent System for Automatic Whitefly Detection in Tomato Greenhouses”. In: *2023 6th Experiment@ International Conference (exp.at'23)*. Évora, Portugal: IEEE, June 2023, pp. 29–30. DOI: 10.1109/exp.at2358782.2023.10546195.

[14] Xiaolei Zhang et al. “Automatic pest identification system in the greenhouse based on deep learning and machine vision”. In: *Frontiers in Plant Science* 14 (Sept. 2023), p. 1255719. DOI: 10.3389/fpls.2023.1255719.

[15] Song Wang et al. “A Deep-Learning-Based Detection Method for Small Target Tomato Pests in Insect Traps”. In: *Agronomy* 14.12 (Dec. 2024). Publisher: MDPI AG, p. 2887. DOI: 10.3390/agronomy14122887.

[16] Jianyu Shi et al. “Small Target Insect Detection Based on Improved YOLOv8n”. In: *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Hyderabad, India: IEEE, Apr. 2025, pp. 1–5. DOI: 10.1109/icassp49660.2025.10890801.

[17] Dan Jeric Arcega Rustia et al. “Online semi-supervised learning applied to an automated insect pest monitoring system”. In: *Biosystems Engineering* 208 (Aug. 2021), pp. 28–44. DOI: 10.1016/j.biosystemseng.2021.05.006.

[18] Xianwei Li et al. “MATeacher: Multi-scale Views Learning and Adaptive Unsupervised Weight for Semi-supervised Pest Region Detection”. In: *2024 6th International Conference on Robotics, Intelligent Control and Artificial Intelligence (RICAI)*. Nanjing, China: IEEE, Dec. 2024, pp. 698–703. DOI: 10.1109/RICAI64321.2024.10911445.

[19] Paweł Majewski et al. “Improved Pest Detection in Insect Larvae Rearing with Pseudo-Labelling and Spatio-Temporal Masking:” in: *Proceedings of the 19th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*. Rome, Italy: SCITEPRESS - Science and Technology Publications, 2024, pp. 349–356. DOI: 10.5220/0012311300003660.

[20] Jiale Zhou et al. “Mutual learning with memory for semi-supervised pest detection”. In: *Frontiers in Plant Science* 15 (June 2024), p. 1369696. DOI: 10.3389/fpls.2024.1369696.

[21] Cheng Zhou and Fanhuai Shi. “Perceptive Teacher: Semi-Supervised small object detection of Brassica Chinensis seedlings and pest infestations”. In: *Computers and Electronics in Agriculture* 229 (Feb. 2025), p. 109956. DOI: 10.1016/j.compag.2025.109956.

[22] Laura Gomez-Zamanillo et al. “Semi-Supervised Approach for Automatic Counting of Whiteflies With Small Annotated Dataset”. In: *IEEE Access* 13 (2025), pp. 55586–55598. DOI: 10.1109/ACCESS.2025.3552195.

[23] L. Minh Dang et al. “An efficient zero-labeling segmentation approach for pest monitoring on smartphone-based images”. In: *European Journal of Agronomy* 160 (Oct. 2024), p. 127331. DOI: 10.1016/j.eja.2024.127331.

[24] Waldemar Raaz et al. “Smart Checkpots–Proof of concept for a mobile pest and climate monitoring system in greenhouses”. In: *DGG-Proceedings* 11.8 (2023). DOI: 10.5288/DGG-PR-11-08-WR-2023.

[25] Alexander E. Dearden et al. “Visual modelling can optimise the appearance and capture efficiency of sticky traps used to manage insect pests”. In: *Journal of Pest Science* 97 (2024), pp. 469–479. DOI: https://doi.org/10.1007/s10340-023-01604-w.

[26] *YOLOv8*. URL: https://docs.ultralytics.com/models/yolov8/.

[27] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. “YOLOv4: Optimal Speed and Accuracy of Object Detection”. In: *arXiv preprint arXiv:2004.10934* (2020).

[28] Hongyi Zhang et al. “mixup: Beyond Empirical Risk Minimization”. In: *International Conference on Learning Representations*. 2018.

[29] Ekin D. Cubuk et al. “RandAugment: Practical Automated Data Augmentation with a Reduced Search Space”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. 2020.

[30] Zhun Zhong et al. “Random Erasing Data Augmentation”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2020.

[31] Satoshi Suzuki and Keiichi Abe. "Topological structural analysis of digitized binary images by border following". In: *Computer Vision, Graphics, and Image Processing* 30.1 (1985), pp. 32–46.

[32] David H. Douglas and Thomas K. Peucker. "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature". In: *Cartographica: The International Journal for Geographic Information and Geovisualization* 10.2 (1973), pp. 112–122.

[33] Martin A. Fischler and Robert C. Bolles. "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography". In: *Communications of the ACM* 24.6 (1981), pp. 381–395.

[34] Richard O. Duda and Peter E. Hart. "Use of the Hough transformation to detect lines and curves in pictures". In: *Communications of the ACM* 15.1 (1972), pp. 11–15.

[35] Nobuyuki Otsu. "A threshold selection method from gray-level histograms". In: *IEEE Transactions on Systems, Man, and Cybernetics* 9.1 (1979), pp. 62–66.

[36] Dong-Hyun Lee et al. "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks". In: *Workshop on challenges in representation learning, ICML* 3.2 (2013), p. 896.